Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Seminer sunumları
  - Bu ders de öğrenci seminer sunumlarına ayrıldı. Her sunumda bir öğrenci veri sıkıştırmayla ilgili yayımlanmış bir makaleyi yaklaşık 15 dakikada anlattı; hoca sunum sırasında ve sonrasında sorular sordu.
  - Sekiz sunum yapıldı: evrişimsel sinir ağlarının (CNN) sıkıştırılması, DeepZip (RNN ile kayıpsız sıkıştırma), dinamik v2vDC sözlük sıkıştırıcısı, FastText.zip (metin sınıflandırma modellerinin sıkıştırılması), IDBE (akıllı sözlük tabanlı kodlama), BERT tabanlı kısa metin sıkıştırma, öğrenilmiş sinirsel yinelemeli kod çözücü ve RNN ile tam çözünürlüklü görüntü sıkıştırma.
- Sunum 1: CNN'lerin sıkıştırılması (pruning, quantization, Huffman)
  - CNN'de girdi, yapay sinir ağına (YSA) girmeden önce evrişim (convolution) ve ortaklama (pooling) katmanlarından geçer; amaç YSA'ya daha küçük boyutlu veri vermektir. Ortaklama maksimum ya da ortalama olabilir (ekrandan, 00:10:00).
  - Doğal dil işlemede cümle, one-hot kodlamayla bir matrise dönüştürülür; her kelime sözlükteki sırasına karşılık gelen sütunda 1 alır.
  - Sıkıştırmanın iki avantajı vardır: hesaplama yükü azalır, bellek daha verimli kullanılır. Dezavantajı: parametreler yanlış seçilirse doğruluk (accuracy) büyük ölçüde düşer.
  - Pruning (budama): eşik değerin (makalede 0,02) altındaki ağırlıklar, seyrek etkinleşen düğümler ya da filtreler ağdan silinir.
  - Quantization (nicemleme): sürekli (float) değerler tamsayıya çevrilir. Slayttaki formüller (ekrandan, 00:18:40): $Q(x) = \lfloor x/\text{scale} + \text{zero point} \rfloor$, scale $= \dfrac{f_{max} - f_{min}}{q_{max} - q_{min}}$.
  - İki türü vardır: eğitim sonrası nicemleme (post-training quantization) ve nicemlemeye duyarlı eğitim (quantization aware training); ikincisinde her geri yayılımdan sonra yeniden nicemleme yapılır ve doğruluk daha az düşer.
  - Huffman, nicemlenmiş tamsayı ağırlıkları kodlar. Üç adım birlikte kullanılır; slayttaki şemada pruning 9–13 kat, nicemleme ile 27–31 kat, Huffman ile 35–49 kat küçülme, doğruluk aynı (ekrandan, 00:20:00).
- Sunum 2: DeepZip, RNN ile kayıpsız sıkıştırma
  - RNN geçmiş verilerden yeni veriyi tahmin eder; uzun dizilerde kaybolan gradyan sorunu yaşanır. LSTM (hücre durumu ve kapılar, sigmoid ile 0–1 arası) ve GRU (sıfırlama ve güncelleme kapısı) bu sorunu azaltır.
  - DeepZip, RNN tabanlı bir olasılık tahmincisi ile aritmetik kodlayıcıdan oluşur. Tahminci softmax katmanıyla bir sonraki sembolün olasılığını verir; aritmetik kodlayıcı 0–1 aralığını bu olasılıklara göre daraltır. Kod çözücü aynı tahminleri ters yönde kullanır.
  - Model az epoch eğitildiyse tahmincinin ağırlıkları da saklanmalıdır ve sıkıştırılmış boyuta eklenir.
  - Entropisi bilinen yapay veriler (IID, Markov-k) ve gerçek veriler (Wikipedia metni, DNA, sözde rastgele sayı dizileri) üzerinde denendi. 16 hücreli DeepZip GRU modeli IID veride birkaç yinelemede entropi sınırına ulaşır (ekrandan, 00:30:00).
  - Sözde rastgele (PRNG) dizilerde Gzip kötü sonuç verirken DeepZip bu dizilerin rastgele olmadığını yakalayıp sıkıştırabildi; DNA'da özel sıkıştırıcı MFCompress'ten iyi, metinde ise çok uzun bağımlılık olmadığından daha zayıf sonuç verdi.
- Sunum 3: Dinamik v2vDC (kelime tabanlı sözlük sıkıştırıcı)

  - Metni kelime ve kelime dizilerine ayırıp (parse) her birine bir kod kelimesi (codeword) atar; sıkıştırma oranını yaklaşık %10 artırır ve sıkıştırılmış metin üzerinde arama yapılabilir (ekrandan, 00:40:00).
  - Dinamik sürüm sözlüğü her kelimeden sonra günceller; gönderici ve alıcı eşzamanlı çalışır, verinin tamamını beklemeye gerek kalmaz.
  - Terminal sembol yeni kelime için üretilir; terminal olmayan (non-terminal) sembol, yeni sembolü bir önceki terminal sembolle birleştirir. "the more I know about you" için 11 sembol oluşur: the, more, the more, I, more I, know, I know, about, know about, you, about you (ekrandan, 00:43:40).
  - Sonuç tablosunda ALL veri setinde v2vdc (lzma, kelime) %22,81, p7zip %22,80 sıkıştırma oranı verdi (ekrandan, 00:46:40). Açma (decompress) tarafında p7zip'ten hızlıdır; en büyük dezavantajı çok bellek gerektirmesidir.
- Sunum 4: FastText.zip, metin sınıflandırma modellerinin sıkıştırılması
  - FastText kelimeleri karakter n-gramlarına bölerek öğrenir; n-gram sayısı çok olduğundan modeller büyüktür. Amaç doğruluk biraz düşse de modeli çok küçültmektir.
  - Ürün nicemleme (product quantization, PQ): gerçek vektörler önceden tanımlı bir kümedeki en yakın merkez vektöre yaklaştırılır. İkilileştirme (binarization) eşiğe göre 0/1 verir; nicemleme birden çok seviye kullanır.
  - Nicemlemeden sonra yeniden eğitim (retraining), sık geçen ama ayırt edici olmayan kelimelerin budanması ve hashing kullanıldı. Bloom filtresi bellek kazandırır ama pratikte performansı düşürdüğü için tercih edilmedi (ekrandan, 01:00:00).
  - Sonuç: 64 KiB modelde ortalama doğruluk kaybı %0,8, 32 KiB'de %1,7, 16 KiB'de %3,5; sıkıştırma ×1000–4000 (ekrandan, 01:01:35).
- Sunum 5: IDBE, akıllı sözlük tabanlı kodlama
  - Burrows-Wheeler dönüşümünden (BWT) esinlenir; metni önceden hazırlanmış bir sözlükle daha iyi sıkıştırılabilir bir ara forma dönüştürür. Benzer çalışmalar: `BWT < input | MTF | RLE | ARI > output` zinciri ve star encoding (ekrandan, 01:20:00).
  - Algoritma: sözlüğü tabloya oku; girdiden token oluştur; bir karakterden uzun token tabloda yoksa olduğu gibi yaz, varsa kod uzunluğunu (ASCII 251 = uzunluk 1, 252 = uzunluk 2 …) kodun önüne koyup yaz; boşluk yoksa ASCII 255 işaretçisi ekle (ekrandan, 01:23:40).
  - Calgary korpusunda bib dosyası için BPC: BWT 2,11, BWT + *Encode 1,93, BWT + IDBE 1,69 (ekrandan, 01:26:35).
- Sunum 6: BERT tabanlı kısa metin sıkıştırma
  - BERT, cümleyi iki yönlü değerlendiren, maskeli dil modeli (kelimelerin %15'i seçilir; bunların %80'i maske, %10'u rastgele kelime, %10'u aynen) ve sonraki cümle tahmini (NSP) ile eğitilen bir modeldir.
  - Sıkıştırma: pencere içindeki kelimelerden sonra gelen kelime BERT'in tahmin listesinde varsa listedeki indisi I dizisine yazılır; yoksa I'ya 0 yazılır ve kelime D dizisine eklenir. I dizisi FSE, D dizisi Huffman ile kodlanır (ekrandan, 01:38:35; 01:40:00).
  - Gzip, Bzip2, Zstd, Smaz, Shoco ve Shoco TR ile karşılaştırıldı; çok kısa metinlerde genel sıkıştırıcılar veriyi büyütür (%100'ün üzeri), BERT yöntemi kısa metinlerde en iyi sonuçları verdi (ekrandan, 01:42:40).
- Sunum 7: Kayıplı görüntü sıkıştırmada öğrenilmiş sinirsel yinelemeli kod çözücü
  - Soru: verilen bir kodlayıcı ve nicemleme şeması için en iyi kod çözücü nasıl olur? JPEG (8×8 DCT) ve JPEG 2000 (64×64, DWT, uyarlamalı ikili aritmetik kodlayıcı) temel alınır (ekrandan, 01:50:00).
  - Kod çözücü, komşu blokların bilgisini kullanan bir RNN (Elman RNN) ile bloğu yinelemeli olarak yeniden kurar; slayttaki şekil 2 bölüm (episode) ve bölüm başına 3 adım gösterir (ekrandan, 01:53:10).
  - Kayıp fonksiyonu MAE ve MSE'nin dışbükey birleşimidir: $D = (1-\alpha)D_{MAE} + \alpha D_{MSE}$, $\alpha = 0{,}235$ (ekrandan, 01:56:10).
- Sunum 8: RNN ile tam çözünürlüklü görüntü sıkıştırma (İngilizce sunum)
  - Kodlayıcı, ikilileştirici (binarizer) ve kod çözücüden oluşan yinelemeli ağ, yeniden eğitim gerektirmeden değişken sıkıştırma oranı sağlar.
  - Klasik kodlama zinciri: kaynak görüntü (8×8 bloklar) → kaynak kodlayıcı (DFT/DCT/DWT, kayıpsız) → nicemleyici (yüksek frekansları atar, kayıplı) → entropi kodlayıcı (Huffman, kayıpsız) → sıkıştırılmış sinyal (ekrandan, 02:10:00).
  - GRU, artık (residual) GRU, LSTM ve ilişkisel (associative) LSTM; tek atış (one-shot), toplamalı (additive) ve ölçeklenmiş artık (additive rescaled residual) yeniden kurma

denendi. Ölçütler MS-SSIM ve PSNR-HVS; 32×32 eğitim verisiyle en iyi sonucu GRU (one-shot) verdi (ekrandan, 02:30:00).

Hocanın Özellikle Vurguladığı Kısımlar

- Sıkıştırılmış veri üzerinde arama
  - Hoca v2vDC sunumunda sıkıştırılmış veri üzerinde mi arama yapıldığını sordu; yöntem kelime düzeyinde kod kelimeleri kullandığından sıkıştırılmış metin üzerinde arama yapılabilir.
- Gönderici ve alıcının eşzamanlılığı
  - Hoca v2vDC tablosunu adım adım okuttu; iki sembol arasında bırakılan boşluğun alıcıya yeni birleşik sembol eklendiğini bildirdiği netleşti.
- Sözlüğe ne konduğu açık olmalıdır
  - IDBE sunumunda hoca, tabloya kelimenin doğrusunun mu yanlışının mı konduğunun makaleden anlaşılmadığını belirtti; harfi eksik kelime örneği ile algoritma arasındaki bağlantıyı sorguladı.
- BWT'nin sezgisi
  - Hoca BWT'yi, bir resmi 256 renge indirip sadeleştirmeye benzetti: BWT karakterlerin yerini, arka arkaya gelen aynı karakterler çoğalacak biçimde değiştirir ve sıkıştırmayı kolaylaştırır.
- Makale künyesi ve güncelliği
  - Hoca her sunumda makalenin yılını sordu ve son yıllara ait çalışmaların seçilmesini önemsedi.

Kısa Tekrar Notları

- CNN sıkıştırma: pruning (ağırlık sayısını azaltır) → quantization (ağırlık başına bit sayısını azaltır) → Huffman (kodlar); doğruluk korunarak 35–49 kata kadar küçülme.
- Nicemleme: scale $= (f_{max} - f_{min})/(q_{max} - q_{min})$, $Q(x) = \lfloor x/\text{scale} + \text{zero point} \rfloor$.
- DeepZip = RNN olasılık tahmincisi + aritmetik kodlayıcı.
- v2vDC: terminal sembol yeni kelime, non-terminal sembol önceki sembol + yeni sembol.
- FastText.zip: PQ + budama + hashing; 64 KiB'de %0,8 doğruluk kaybı.
- IDBE: sözlükteki kelime yerine "uzunluk işaretçisi (251–254) + kod"; BWT ile birlikte kullanılır.
- BERT sıkıştırma: tahmin edilen kelime → indis (I, FSE), tahmin edilemeyen kelime → D dizisi (Huffman).
- Öğrenilmiş kod çözücü: $D = (1-\alpha)D_{MAE} + \alpha D_{MSE}$, $\alpha = 0{,}235$.
- Görüntü kodlama zinciri: dönüşüm (kayıpsız) → nicemleme (kayıplı) → entropi kodlama (kayıpsız).

Detaylı Açıklamalar

- CNN sıkıştırması üç tamamlayıcı adımdan oluşur. Budama, eşik altındaki bağlantıları ya da seyrek etkinleşen düğümleri siler; bağlantı silmek mimariyi değiştirmez, düğüm silmek karmaşıklığı daha çok azaltır ama yanlış eşikte doğruluğu daha çok bozar. Nicemleme, 32 bitlik float ağırlıkları 8 bitlik tamsayılara eşler (FP32 → INT8); eşleme bir ölçek çarpanı ve sıfır noktası ile yapılır. Nicemlenmiş tamsayıların dağılımı dengesiz olduğundan Huffman kodlaması belleği bir kez daha küçültür. Sunumdaki BERT ve TensorFlow tablolarında nicemlemeye duyarlı eğitimde doğruluğun eğitim sonrası nicemlemeye göre daha az düştüğü gösterildi.
  - Ders sonrası not: Slayttaki INT8 ekseninde uçlar -127 ve -128 diye yazılmıştı; INT8 aralığı büyük olasılıkla -128 ile 127 olmalıdır. Slayttaki sıfır noktası formülü $(f_{min} + f_{max})/2$ da yaygın tanımdan (zero point $= q_{min} - f_{min}/\text{scale}$) farklıdır (ekrandan, 00:18:40).
- DeepZip'te sıkıştırma kalitesi tahmincinin ne kadar iyi tahmin ettiğine bağlıdır: tahmin edilen olasılık gerçeğe yakınsa aritmetik kodlayıcı az bit harcar. Markov-k kaynaklarında k büyüdükçe RNN'in bağımlılığı yakalaması zorlaşır; dizi uzunluğu 8 ile eğitilen GRU, dizi uzunluğunun yaklaşık iki katına kadar bağımlılıkları yakalayabildi.

- v2vDC'de sözlük kullanım sıklığına göre sıralanır; bir sembolün sıklığı değişip sırası değişince kod kelimesi de değişir. Kod kelimeleri için yoğun bir önek kod kullanıldığı söylendi.
  - Ders sonrası not: Öğrenci sonucu "%23, yani 100 MB veri 23 MB'ye düştü" diye açıkladı; tablodaki değerler sıkıştırılmış boyutun düz metin boyutuna oranıdır, yani küçük değer iyi sıkıştırmadır.
- FastText.zip'te performans kaybının kabul edildiği, asıl amacın model boyutu olduğu vurgulandı. Hashing her yeni değerin sözlükte olup olmadığını O(1) karmaşıklıkta denetler; çok yoğun yükte bellek kullanımı artar. Bloom filtresi değerin geçmediğini kesin söyleyebilir ama geçtiğini kesin söyleyemez.
- IDBE'nin sonuç tablosunda çoğu Calgary dosyasında BWT + IDBE en düşük BPC'yi verdi.
  - Ders sonrası not: Öğrenci IDBE'nin her dosyada daha iyi olduğunu söyledi; tabloya göre geo (5,18'e karşı 4,84) ve paper1 (2,26'ya karşı 1,59) dosyalarında *Encode daha iyidir. 14 dosyanın 12'sinde IDBE önde, ortalama BPC: BWT 2,78,* Encode 2,52, IDBE 2,40 (python ile hesaplandı; ekrandan, 01:26:35).
- BERT tabanlı yöntemde pencere boyutu (W = 5, 10, 15) ve tahmin listesi uzunluğu (P = 64, 128, 254) değiştirilerek denendi. Çok kısa metinlerde (0–99 bayt) Gzip %165,4, Bzip2 %228,2 oranıyla veriyi büyüttü; BERT en iyi %102,8'e ulaştı. Metin uzadıkça oranlar düşer (900 bayt üzeri, W = 15, P = 254 için %41,7) (ekrandan, 01:42:40). Hoca Smaz ve Shoco'yu sordu; Smaz küçük harfli İngilizce metinler, HTML ve URL'ler için statik sözlük içerir, Shoco TR Türkçe metinde başarılıdır, yani bu yöntemler dile bağımlıdır.
- Öğrenilmiş yinelemeli kod çözücüde yalnız MSE ile eğitim JPEG'e göre yetersiz PSNR kazancı verdiği için MAE eklenmiştir. Eğitimde görüntüler her epoch başında karıştırılır ve tarama başlangıç köşesi rastgele seçilir (ekrandan, 02:00:00).
  - Ders sonrası not: Hoca kayıp fonksiyonundaki K ve B'nin blok boyutları olup olmadığını sordu, öğrenci onayladı. Slayta göre K büyük olasılıkla bir bölümdeki yeniden kurma adımı sayısı, B ise mini-batch'teki hedef görüntü parçası sayısıdır (ekrandan, 01:53:10; 01:56:10).
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.